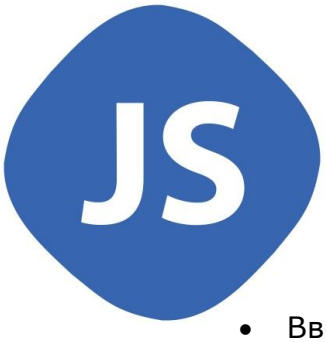


Объект FormData, 1

- Введение
- Доступ к элементам форм
- Создание объекта FormData
- Функция Array.from()
- Методы объекта FormData
- Итерация по свойствам объекта FormData

Введение

Форма (form) в HTML — раздел документа, позволяющий пользователю вводить информацию для последующей обработки системой.

Создание HTML-форм – неотъемлемая часть сайта и важный навык веб-разработчика. Синтаксически форма в HTML задаётся с помощью элемента <**form**> и в дополнение к разметке обычных элементов содержит специальную разметку для **элементов управления** (controls).

Элементы управления веб-форм — это текстовые поля, кнопки, списки, чекбоксы и другие элементы, которые активизируются кликом мыши. **Технически формы передают данные от пользователя удаленному серверу**.

Для получения и обработки данных форм как правило используются серверные языки веб-программирования, такие как PHP, Perl, Python и др.

До появления стандарта HTML-5 для стилизации форм в разных браузерах приходилось прилагать немало усилий. Кроме того, формы требовали применения JavaScript для проверки введенных данных, а также были лишены специфических типов полей ввода для указания повседневной информации типа дат, адресов электронной почты или URL-адресов.





HTML5-формы решили большинство этих распространенных проблем благодаря наличию новых атрибутов, предоставив возможность изменять внешний вид элементов форм за счет CSS3.

Элементы управления форм

Элементы формы HTML, которые используются для создания элементов управления взаимодействием пользователя в браузере, называются **элементами управления формами**.

Элементы управления позволяют пользователям вводить информацию для ее обработки на стороне сервера. Характер взаимодействия с сервером может варьироваться в зависимости от типа элемента управления, используемого при создании формы. Например, переключатели обычно используются для приема информации о поле.

Существуют различные типы элементов управления формами, которые разработчик можем использовать для сбора данных.

Типы элементов	Описание
<**input type="text**">	Текстовый ввод обычно используется для приема символов от пользователя
<**input type="number**">	Тип ввода **number**, используется для приема числовых данных от пользователя
<**input type="checkbox**">	Входные **флажки** отображаются в виде квадратных рамок, которые можно установить или снять в зависимости от требований пользователя.
<**input type="radio**">	Радиовход определяет радиокнопку для выбора одного из множества вариантов.
<**select**>	Тег **select** используется для создания выпадающего списка в форме
<**input type="datetime-local**">	Тип ввода **datetime** определяет графический интерфейс для выбора месяца
<**input type="date**">	Тип ввода **date** определяет графический интерфейс для выбора даты
<**input type="month**">	Тип ввода **month** определяет графический интерфейс для выбора месяца

<input type="week">	Тип ввода **week** определяет графический интерфейс для выбора дня недели
<input type="time">	Тип ввода **time** определяет графический интерфейс для выбора времени.
<input type="range">	Диапазон типов ввода, используется для определения ползункового элемента управления для ввода числа
<input type="email">	Тип ввода **email** используется для определения области ввода, которая принимает электронную почту
<input type="url">	Тип ввода **url** используется для определения области ввода, которая принимает URL

Примечание. Подробнее о формах и стилизации форм в нашем курсе **HTML&CSS**.

Доступ к элементам форм

В JavaScript, существует много (слишком много) способов сделать одно и то же действие. Получить доступ к элементу управления формы можно напрямую, можно через ссылку на саму форму.

Тестируем получение ссылки на **форму**.

example_1.

```
<script>

    // тестируем способы получения доступа к форме

    var form = document.forms;

    console.log(form);

    // console.log(form[0]);

    var form = document.getElementsByName("frm");
```



```
    console.log(form);
    // console.log(form.item(0));
    var form = document.querySelector("#frm");
    console.log(form);
    var form = document.getElementById("frm");
    console.log(form);
    // и др.
</script>
```

Разберем некоторые способы получения ссылки на элемент управления формы. Например, в следующем HTML-коде формы (example_2) есть текстовое поле:

```
<form name="frm">
    <input type="text" name="el" id="el">
</form>
// рассмотрим некоторые из возможных способов доступа к полю
<script>
    // 1. прямой доступ по id
    console.log(el);
    // 2. дотуп как к коллекции элементов формы
    console.log(document.forms[0].elements[0]);
    // 3. доступ как к свойству узла формы
    console.log(document.frm.el);
    // 4. JS-метод getElementById()
    console.log(document.getElementById('el'));
    // 5. JS-метод getElementsByName()
    console.log(document.getElementsByName('frm')[0].el);
    // и другие ...
```





```
</script>
```

Способ (1) слишком общий, чтобы быть полезным и применяется, как правило, в режиме тестирования.

Способ (3) кажется наиболее интуитивно понятным и достаточно широко используется, но может вызывать проблемы совместимости и кроссбраузерности.

Способ (5) – избыточно громоздкий.

Способы (2) и (4) хорошо документированы и гарантированно будут работать в любом браузере. При этом способ (2) интуитивно менее нагляден, способ (4) потребует как атрибут **id** (для выбора), так и атрибут **name** для отправки данных в серверный скрипт.

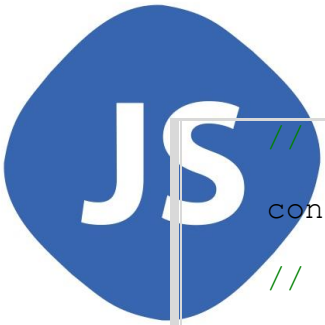
> ***Примечание***. Для того, чтобы данные формы были отправлены в серверный сценарий, атрибут **name** элемента управления формы – **обязателен**.

В идеале было бы лучше иметь или атрибут **id** или атрибут **name**, наличие обоих несколько **избыточно**, особенно если идентификатор не используется в файлах стилей (а он не должен там использоваться).

Тестируем получения ссылки на **элемент управления** формы.

example_2.

```
<form name="frm">
    <input type="text" name="el" id="el">
</form>
<script>
    // тестируем способы получения доступа к элементу формы
    // 1. прямой доступ по id
    console.log(el);
```



```
    // 2. дотуп как к коллекции элементов формы
    console.log(document.forms[0].elements[0]);
    // 3. доступ как к свойству узла формы
    console.log(document.frm.el);
    // 4. JS-метод getElementById()
    console.log(document.getElementById('el'));
    // 5. JS-метод getElementsByName()
    console.log(document.getElementsByName('frm')[0].el);
    // и другие ...
</script>
```

Лучшую практику применения придется выбрать самостоятельно.

> ***Примечание***. Выработайте любимый способ выполнения доступа к элементам формы так же, как того не замечая, вы выработали любимый способ перебора элементов коллекций (массивов ;)

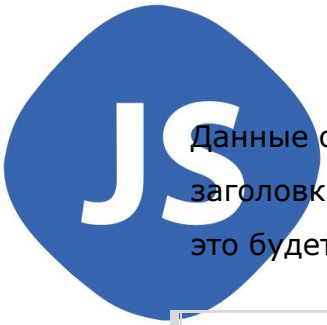
Создание объекта FormData

Объект **FormData** это объект, представляющий данные HTML формы.

Синтаксис создания объекта:

```
let formData = new FormData([form]);
```

Если передать в конструктор элемент HTML-формы form, то создаваемый объект автоматически прочитает из неё поля. Особенность объекта **FormData** заключается в том, что методы для работы с сетью позволяют указать объект **FormData** в свойстве тела запроса **body**.



Данные объекта будут соответствующим образом закодированы и отправлены с заголовком **Content-Type: multipart/form-data**. Таким образом, для сервера это будет выглядеть как обычная отправка формы.

> ***Примечание***. Отправке и получению данных посвящена следующая тема: **Асинхронный JavaScript**.

Создадим объект **FormData**.

example_3.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="12345"><p>
    <input type="submit">
</form>
<script>
    // ссылка на форму
    let form = document.querySelector("#frm");
    console.log(form);
    // создание объекта FormData
    let form_data = new FormData(form);
    console.log(form_data);
</script>
```

Для дальнейшей работы нам понадобится функция **Array.from**().



Функция Array.from()

В JavaScript и браузерных API есть много объектов, которые очень похожи на массив, но не являются им. Объекты могут выглядеть как массив, но не иметь всех методов массива: **forEach**(), **map**(), **filter**() и так далее.

Такие объекты приходится превращать в массивы для удобства работы с ними или для интеграции с библиотеками. **Array.from**() создан, чтобы решить проблему конвертации таких объектов в новый массив.

Array.from() — это функция в JavaScript, которая создаёт **новый массив на основе переданного объекта**.

Функция **Array.from**() принимает три аргумента, последние два из которых не обязательны:

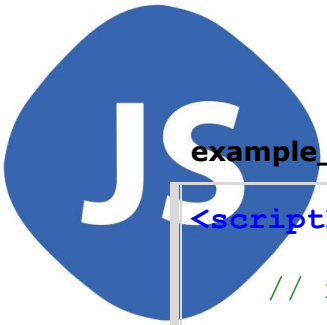
- Объект, на основе которого создаётся массив.
- Функция преобразования элемента перед его добавлением в массив. Работает как метод **map**().
- Значение, которое будет использоваться как **this** в функции из второго параметра.

Функция возвращает новый массив, составленный из элементов переданного объекта.

Array.from() работает не со всеми объектами. Объект должен обладать одним из двух свойств, чтобы его получилось превратить в массив:

- Элементы объекта проиндексированы и объект имеет свойство **length**. Такие объекты называют массивоподобными, потому что именно эти свойства присущи массиву. Этим свойством обладают объекты **arguments**, **NodeList**, **HTMLCollection**.
- Объект **итерируемый**, то есть реализует интерфейс **Iterable**. Этим свойством обладают объекты **Set** и **Map**.

В демонстрационном примере **example_4** попытаемся преобразовать к массиву пользовательский объект.



example_4.

```
<script>
    // пользовательский объект
    let user = {
        login: 'DenisDream',
        email: 'denis-dream@gmail.com',
        pwd: 'qwerty@'
    }
    // объект
    console.log(user);
    // пустой массив
    console.log(Array.from(user));
</script>
```

Преобразование к массиву объектов **Map** и **Set**.

example_5.

```
<script>
    // пользовательский объект
    let user = {
        login: 'DenisDream',
        email: 'denis-dream@gmail.com',
        pwd: 'qwerty@'
    }
    // объект Map
    user_map = new Map();
    // добавляем пользовательский объект в объект Map
```

```
    user_map.set('user', user);
    user_arr = Array.from(user_map); // Array
    console.log(user_arr);
    // объект Set
    user_set = new Set();
    // добавляем пользовательский объект в объект Set
    user_set.add(user);
    user_arr = Array.from(user_set); // Array
    console.log(user_arr);
</script>
```

Преобразование к массиву коллекций **NodeList** и **HTMLCollection**.

example_6.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
    // ссылка на форму
    var form = document.getElementById("frm");
    console.log(form.elements); // HTMLFormControlsCollection
    console.log(Array.from(form.elements));
    // console.log(form.elements[2]);
```

```
    console.log(form.children); // HTMLCollection
    console.log(Array.from(form.children));
    // console.log(form.children[2]);
    // ссылка на элементы input
    inps = form.querySelectorAll("form input");
    console.log(inps); // NodeList
    console.log(Array.from(inps));
    // console.log(inps.item(2));
</script>
```

Преобразование к массиву объекта **FormData**.

example_7.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
    // объект FormData
    let form_data = new FormData (form);
    console.log(form_data); // FormData
    console.log(Array.from(form_data)); // Array
</script>
```

Еще раз. Особенность объекта **FormData** заключается в том, что методы для работы с сетью позволяют указать объект **FormData** в свойстве тела запроса **body**.

При создании массива происходит **поверхностное копирование элементов**. Если исходный объект содержит итерируемые элементы, являющиеся объектами, то эти объекты будут скопированы по ссылке.

example_8.

```
<script>
    // пользовательский объект
    let user = {
        login: 'DenisDream',
        email: 'denis-dream@gmail.com',
        pwd: 'qwerty@',
        course: ['python', 'php']
    }
    // объект Set
    user_set = new Set();
    user_set.add(user);
    // изменяем свойство пользовательского объекта
    user.course.push('javascript');
    // измененилтсь оба объекта
    console.log(user);
    console.log(user_set);
</script>
```

Возникает резонный вопрос, зачем преобразовывать и **без того итерируемые объекты** и **коллекции** к массиву. Еще раз повторю, в JavaScript, существует



много способов сделать одно и то же действие. Иногда работа с массивом будет удобнее работы с объектом. Например, при **сериализации** (представлению к строке) элементов формы.

example_9.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
    // ссылка на форму
    let form = document.getElementById("frm");
    form_data = new FormData(form);
    // сериализуем объект
    console.log(JSON.stringify(form_data));
    // преобразуем объект FormData в массив
    form_data_arr = Array.from(form_data);
    // сериализуем массив
    console.log(JSON.stringify(form_data_arr));
</script>
```

Примечание. Дополнительная **практика** работы с формами будет представлена в следующем уроке.



Методы объекта FormData

С помощью указанных ниже **методов** мы можем изменять поля в объекте **FormData**:

- **append**(name, value) – добавляет к объекту поле с именем name и значением value;
- **set**(name, value) – добавляет к объекту поле с именем name и значением value. О разнице между **append**() и **set**() далее;
- **append**(name, blob, fileName) – добавляет поле, как будто в форме имеется элемент <input type="file">, третий аргумент fileName устанавливает имя файла (не имя поля формы), как будто это имя из файловой системы пользователя;
- **delete**(name) – удаляет поле с заданным именем name;
- **get**(name) – получает значение поля с именем name;
- **has**(name) – если существует поле с именем name, то возвращает **true**, иначе **false**.

Технически форма может иметь много полей с одним и тем же именем **name**, поэтому несколько вызовов **append** добавят несколько полей с одинаковыми именами.

Разница между **set**() и **append**() в том, что **set** удаляет все уже имеющиеся поля с именем name и только затем добавляет новое. То есть этот метод гарантирует, что будет существовать только одно поле с именем name, в остальном он аналогичен методу **append**.

Тестируем метод **get**().

example_10.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
```

```
    <input type="submit">
</form>
<script>
    // ссылка на форму
    let form = document.querySelector("#frm");
    // преобразование формы в объект FormData
    let form_data = new FormData (form);
    // получение значений элементов формы
    let login = form_data.get("login");
    let email = form_data.get("email");
    let pwd = form_data.get("pwd");
    // вывод данных формы в консоль
    console.log("login -> ", login);
    console.log("email -> ", email);
    console.log("pwd -> ", pwd);
</script>
```

Тестируем метод **append**().

example_11.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
```

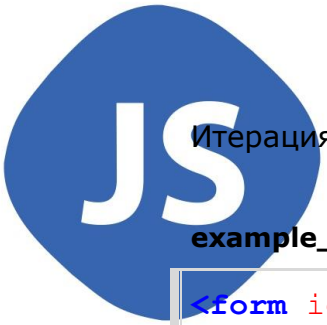
```
    // ссылка на форму
    let form = document.querySelector("#frm");
    // преобразование формы в объект FormData
    let form_data = new FormData (form);
    // добавление к объекту FormData новых параметров
    form_data.append("name", "Денис");
    form_data.append("surname", "Калинин");
    // получение значений элементов формы
    let name = form_data.get("name");
    let surname = form_data.get("surname");
    // вывод данных формы в консоль
    console.log("name -> ", name);
    console.log("surname -> ", surname);
</script>
```

Поля объекта **formData** можно итерировать, используя цикл **for..of**.

Итерация по свойствам объекта FormData

Рассмотрим специальные методы для обхода элементов форм в цикле.

- **keys**() – возвращает **Iterator** , который позволяет пройтись по всем **ключам** для каждой пары ключ-значение , содержащимся внутри объекта **FormData**;
- **values**() – возвращает **iterator** , который позволяет пройтись по всем **значениям** , содержащимся в объекте FormData;
- **entries**() – возвращает **iterator** который позволяет пройтись по всем **парам** ключ-значение, содержащимся внутри объекта FormData;



Итерация **ключей** объекта **FormData**. Метод **keys**().

example_12.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
    // создание объекта FormData
    let form_data = new FormData
    (document.querySelector("#frm"));
    form_data.append("name", "Денис");
    form_data.append("surname", "Калинин");
    console.log(form_data.keys()); // FormData Iterator
    // перебор ключей объекта FormData
    for (const key of form_data.keys()) {
        console.log(key);
    }
</script>
```

Итерация **значений** объекта **FormData**. Метод **values**().

example_13.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
```

```
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
    // создание объекта FormData
    let form_data = new FormData
    (document.querySelector("#frm"));
    form_data.append("name", "Денис");
    form_data.append("surname", "Калинин");
    console.log(form_data.values()); // FormData Iterator
    // перебор значений объекта FormData
    for (const val of form_data.values()) {
        console.log(val);
    }
</script>
```

Итерация **параметров** объекта **FormData**. Метод **entries**().

example_14.

```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
    dream@gmail.com"><p>
    Пароль: <input type="password" name="pwd" value="qwerty@"><p>
    <input type="submit">
</form>
<script>
```

```
    // создание объекта FormData
    let form_data = new FormData
    (document.querySelector("#frm"));
    form_data.append("name", "Денис");
    form_data.append("surname", "Калинин");
    console.log(form_data.entries()); // FormData Iterator
    // перебор параметров объекта FormData
    for (const [key, val] of form_data.entries()) {
        console.log(key, '=>', val);
    }
    // или так
    // перебор параметров объекта FormData
    for (const arr of form_data.entries()) {
        console.log(arr[0], '=>', arr[1]);
    }
    // или так
    // перебор параметров объекта FormData
    for (let [key, value] of form_data) {
        console.log(key, ' -> ', value);
    }
</script>
```

И еще раз про **сериализацию форм**.

example_15.

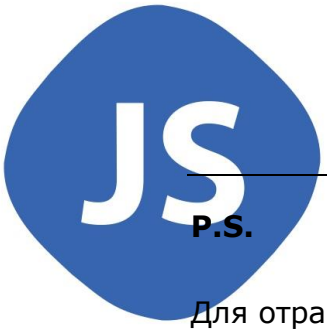
```
<form id="frm">
    Логин: <input type="text" name="login" value="DenisDream"><p>
    E-mail: <input type="email" name="email" value="denis-
```

```
        dream@gmail.com"><p>
        Пароль: <input type="password" name="pwd" value="qwerty@"><p>
        <input type="submit">
</form>
<script>
    // создание объекта FormData
    let form_data = new FormData
    (document.querySelector("#frm"));
    arr1 = Array.from(form_data);
    arr2 = Array.from(form_data.entries());
    // строки аналогичны
    console.log(JSON.stringify(arr1));
    console.log(JSON.stringify(arr2));
</script>
```

Дополнительные примеры и практика в **самостоятельных работах** текущего урока:

- Динамическое создание текстовых элементов управления формы, отправка и обработка данных в целевом файле.
- Сбор м вставка данных формы в скрытое поле. Отправка и обработка данных скрытого поля в целевом файле.
- Сбор и вставка данных формы в атрибут href ссылки. Отправка и обработка данных адресной строки в целевом файле.

Примечание. Каждая самостоятельная работа имеет множество решений. Представленное решение – одно из возможных, и не обязательно совпадет с вашим.



P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**

